

HW2_加分

加分題 A

對 `cycleData_peak.txt` 中的每個基因，自己寫一個函式，依「非缺失值（non-NA）的比例」把資料品質分類：

```
# "High" : non-NA values ≥ 80%
```

```
# "Medium" : non-NA values  $\geq 50\%$  且  $< 80\%$ 
```

```
# "Low" : non-NA values < 50%
```

```
# 基礎作業的 code：讀檔加計算每列不是缺值的數量
data <- read.delim("cycleData_peak.txt") # 讀檔
dim(data) # 觀察維度
count_NA <- function(x){
  length(x) - sum(is.na(x))
}
row_Non_NA_nums <- apply(data, 1, count_NA)
```

完成以下：

1. 把這個函式套用到每一列，統計各品質類別的基因數。

[illegible]

Output:

```
[1] 91.13924 97.46835 98.73418 96.20253 97.46835 94.93671 94.93671 91.13924 96.20253 89.87342 98.73418 92.40506 91.13924 98.73418 96.20253
[16] 93.67089 98.73418 100.00000 100.00000 100.00000 98.73418 100.00000 98.73418 100.00000 98.73418 97.46835 68.35443 70.88608 97.46835 98.73418
[31] 97.46835 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418
[46] 98.73418 100.00000 98.73418 97.46835 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418
[61] 97.46835 89.87342 100.00000 97.46835 94.93671 98.73418 98.73418 100.00000 100.00000 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418
[76] 70.88608 98.73418 98.73418 98.73418 88.60759 97.46835 100.00000 100.00000 97.46835 93.67089 98.73418 98.73418 98.73418 98.73418 98.73418
[91] 98.73418 100.00000 94.93671 98.73418 97.46835 98.73418 100.00000 98.73418 98.73418 93.67089 98.73418 97.46835 97.46835 97.46835 98.73418
[106] 97.46835 97.46835 100.00000 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 97.46835 100.00000 100.00000 98.73418 98.73418
[121] 93.67089 100.00000 98.73418 98.73418 98.73418 97.46835 83.54430 100.00000 93.67089 68.35443 98.73418 93.67089 98.73418 97.46835 98.73418
[136] 98.73418 96.20253 97.46835 97.46835 98.73418 100.00000 100.00000 100.00000 97.46835 97.46835 100.00000 96.20253 97.46835 98.73418 98.73418
[151] 97.46835 97.46835 97.46835 98.73418 98.73418 98.73418 96.20253 98.73418 96.20253 93.67089 94.93671 97.46835 98.73418 98.73418 97.46835
[166] 98.73418 98.73418 98.73418 98.73418 68.35443 97.46835 98.73418 98.73418 98.73418 98.73418 97.46835 94.93671 97.46835 97.46835 100.00000
[181] 98.73418 96.20253 96.20253 98.73418 98.73418 98.73418 98.73418 96.20253 98.73418 97.46835 98.73418 98.73418 98.73418 98.73418 98.73418
[196] 98.73418 98.73418 98.73418 97.46835 96.20253 98.73418 96.20253 97.46835 97.46835 96.20253 98.73418 96.20253 98.73418 97.46835 98.73418
[211] 97.46835 55.69620 97.46835 97.46835 98.73418 96.20253 97.46835 97.46835 56.96203 92.40506 93.67089 94.93671 98.73418 96.20253 83.54430
[226] 78.48101 98.73418 70.88608 97.46835 98.73418 96.20253 98.73418 70.88608 100.00000 96.20253 98.73418 96.20253 98.73418 96.20253 98.73418
[241] 98.73418 97.46835 97.46835 86.07595 94.93671 96.20253 74.68354 98.73418 97.46835 98.73418 98.73418 97.46835 100.00000 98.73418 94.93671
[256] 98.73418 88.60759 98.73418 93.67089 97.46835 96.20253 98.73418 97.46835 98.73418 97.46835 98.73418 100.00000 98.73418 98.73418 96.20253
[271] 97.46835 94.93671 98.73418 98.73418 92.40506 98.73418 98.73418 98.73418 97.46835 87.34177 98.73418 91.13924 98.73418 96.20253 98.73418
[286] 86.07595 97.46835 89.87342 96.20253 97.46835 100.00000 98.73418 98.73418 98.73418 96.20253 83.54430 97.46835 98.73418 97.46835 98.73418
[301] 98.73418 98.73418 97.46835 89.87342 96.20253 97.46835 92.40506 89.87342 98.73418 98.73418 93.67089 97.46835 97.46835 70.88608 98.73418
[316] 86.07595 67.08861 98.73418 98.73418 98.73418 98.73418 100.00000 98.73418 98.73418 86.07595 97.46835 98.73418 97.46835 97.46835 98.73418
[331] 67.08861 96.20253 98.73418 98.73418 98.73418 98.73418 98.73418 97.46835 98.73418 97.46835 98.73418 72.15190 97.46835 97.46835 98.73418
[346] 100.00000 100.00000 98.73418 98.73418 98.73418 98.73418 98.73418 97.46835 98.73418 100.00000 98.73418 100.00000 98.73418 98.73418 97.46835
[361] 98.73418 97.46835 97.46835 98.73418 98.73418 98.73418 96.20253 93.67089 97.46835 100.00000 98.73418 98.73418 98.73418 100.00000 97.46835
[376] 100.00000 96.20253 96.20253 100.00000 100.00000 69.62025 98.73418 86.07595 98.73418 98.73418 98.73418 97.46835 97.46835 100.00000 97.46835
[391] 98.73418 97.46835 100.00000 98.73418 97.46835 98.73418 98.73418 98.73418 98.73418 98.73418 100.00000 98.73418 98.73418 98.73418 98.73418
[406] 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 97.46835 97.46835 97.46835 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418
[421] 97.46835 97.46835 98.73418 97.46835 98.73418 100.00000 24.05063 97.46835 55.69620 98.73418 100.00000 98.73418 98.73418 98.73418 98.73418
[436] 98.73418 98.73418 97.46835 97.46835 98.73418 98.73418 96.20253 98.73418 97.46835 31.64557 98.73418 98.73418 97.46835 98.73418 98.73418
[451] 98.73418 98.73418 100.00000 98.73418 69.62025 97.46835 98.73418 97.46835 87.34177 97.46835 100.00000 98.73418 97.46835 100.00000 97.46835
[466] 83.54430 97.46835 93.67089 98.73418 98.73418 98.73418 94.93671 98.73418 98.73418 81.01266 98.73418 98.73418 98.73418 98.73418 98.73418
[481] 98.73418 97.46835 98.73418 98.73418 96.20253 87.34177 81.01266 98.73418 97.46835 96.20253 96.20253 97.46835 98.73418 97.46835 94.93671
[496] 98.73418 91.13924 67.08861 93.67089 96.20253 98.73418 97.46835 98.73418 98.73418 98.73418 100.00000 98.73418 98.73418 98.73418 98.73418
[511] 98.73418 94.93671 70.88608 98.73418 98.73418 97.46835 98.73418 98.73418 97.46835 98.73418 97.46835 96.20253 98.73418 100.00000 82.27848
[526] 100.00000 67.08861 70.88608 86.07595 94.93671 100.00000 72.15190 98.73418 100.00000 98.73418 97.46835 98.73418 98.73418 98.73418 98.73418
[541] 98.73418 98.73418 78.48101 86.07595 98.73418 96.20253 100.00000 98.73418 98.73418 98.73418 97.46835 98.73418 98.73418 100.00000 98.73418
[556] 98.73418 97.46835 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 81.01266 98.73418 98.73418 98.73418 98.73418 98.73418
[571] 97.46835 97.46835 96.20253 96.20253 92.40506 98.73418 97.46835 98.73418 98.73418 73.41772 98.73418 98.73418 98.73418 96.20253 70.88608
[586] 97.46835 91.13924 100.00000 97.46835 97.46835 98.73418 97.46835 98.73418 91.13924 97.46835 93.67089 97.46835 98.73418 96.20253 98.73418
[601] 100.00000 98.73418 88.60759 100.00000 94.93671 84.81013 96.20253 98.73418 96.20253 97.46835 98.73418 97.46835 98.73418 98.73418 98.73418
[616] 98.73418 98.73418 56.96203 97.46835 97.46835 94.93671 96.20253 96.20253 97.46835 92.40506 98.73418 98.73418 98.73418 98.73418 98.73418
[631] 100.00000 98.73418 96.20253 98.73418 98.73418 100.00000 98.73418 55.69620 98.73418 100.00000 98.73418 98.73418 98.73418 98.73418 98.73418
[646] 98.73418 98.73418 97.46835 98.73418 98.73418 98.73418 96.20253 97.46835 98.73418 98.73418 97.46835 98.73418 97.46835 98.73418 98.73418
[661] 74.68354 98.73418 98.73418 98.73418 98.73418 96.20253 73.41772 94.93671 97.46835 98.73418 94.93671 98.73418 98.73418 97.46835 93.67089
[676] 97.46835 96.20253 98.73418 97.46835 97.46835 94.93671 98.73418 98.73418 100.00000 97.46835 98.73418 98.73418 97.46835 98.73418 98.73418
[691] 98.73418 97.46835 100.00000 97.46835 96.20253 68.35443 98.73418 96.20253 98.73418 100.00000 96.20253 98.73418 98.73418 98.73418 98.73418
[706] 98.73418 100.00000 98.73418 100.00000 100.00000 97.46835 98.73418 97.46835 98.73418 72.15190 67.08861 100.00000 94.93671 98.73418 98.73418
[721] 97.46835 96.20253 97.46835 97.46835 96.20253 98.73418 97.46835 97.46835 98.73418 98.73418 98.73418 98.73418 94.93671 98.73418 98.73418
[736] 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 96.20253 96.20253 98.73418 98.73418 98.73418 98.73418 97.46835 98.73418
[751] 98.73418 98.73418 98.73418 94.93671 97.46835 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 98.73418 100.00000
[766] 97.46835 98.73418 98.73418 98.73418 97.46835 98.73418 97.46835 97.46835 98.73418 96.20253 83.54430 98.73418 98.73418 98.73418 94.93671
[781] 97.46835 98.73418 88.60759 98.73418 94.93671 98.73418 97.46835 96.20253 98.73418 98.73418 98.73418 98.73418 98.73418 96.20253 89.87342
[796] 98.73418 100.00000 100.00000 98.73418 94.93671
```

2. 有多少基因被分為 High、Medium、Low？

```
25 # 2. 有多少基因被分為 High、Medium、Low？
26 count_quality <- list(high = 0, medium = 0, low = 0) # 存放各種品質有幾個
27 quality_list <- list() # 存放哪一列是哪一種品質
28
29 for (i in 1:length(Non_NA_percent)){
30   if (Non_NA_percent[i] >= 80){ # 如果 >= 80%，high的個數加 1
31
32     count_quality[['high']] <- count_quality[['high']] + 1
33     quality_list[[i]] <- 'high'
34
35   }else if(50 <= Non_NA_percent[i] & Non_NA_percent[i] < 80){
36     # 如果 < 80% and >= 50%，medium 的個數加 1
37     count_quality[['medium']] <- count_quality[['medium']] + 1
38     quality_list[[i]] <- 'medium'
39
40   }else{
41     # <50, low 個數加 1
42     count_quality[['low']] <- count_quality[['low']] + 1
43     quality_list[[i]] <- 'low'
44
45   }
46 }
47 t(count_quality)
48 t(quality_list)
```

high medium low

[1,] 757 41 2

[1,] [1,] [2,] [3,] [4,] [5,] [6,] [7,] [8,] [9,] [10,] [11,] [12,] [13,] [14,] [15,] [16,] [17,] [18,] [19,] [20,] [21,]
"high"
[22,] [23,] [24,] [25,] [26,] [27,] [28,] [29,] [30,] [31,] [32,] [33,] [34,] [35,] [36,] [37,] [38,] [39,] [40,] [41,]
"high" "high" "high" "high" "high" "medium" "medium" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high"
[42,] [43,] [44,] [45,] [46,] [47,] [48,] [49,] [50,] [51,] [52,] [53,] [54,] [55,] [56,] [57,] [58,] [59,] [60,] [61,] [62,]
"medium" "medium" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high"
[63,] [64,] [65,] [66,] [67,] [68,] [69,] [70,] [71,] [72,] [73,] [74,] [75,] [76,] [77,] [78,] [79,] [80,] [81,] [82,] [83,]
"high" "high" "high" "high" "high" "high" "high" "high" "high" "medium" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high"
[84,] [85,] [86,] [87,] [88,] [89,] [90,] [91,] [92,] [93,] [94,] [95,] [96,] [97,] [98,] [99,] [100,] [101,] [102,] [103,] [104,]
"high" "high" "high" "high" "high" "medium" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high"
[105,] [106,] [107,] [108,] [109,] [110,] [111,] [112,] [113,] [114,] [115,] [116,] [117,] [118,] [119,] [120,] [121,] [122,] [123,] [124,] [125,]
"high"
[126,] [127,] [128,] [129,] [130,] [131,] [132,] [133,] [134,] [135,] [136,] [137,] [138,] [139,] [140,] [141,] [142,] [143,] [144,] [145,] [146,]
"high"
[147,] [148,] [149,] [150,] [151,] [152,] [153,] [154,] [155,] [156,] [157,] [158,] [159,] [160,] [161,] [162,] [163,] [164,] [165,] [166,] [167,]
"high"
[168,] [169,] [170,] [171,] [172,] [173,] [174,] [175,] [176,] [177,] [178,] [179,] [180,] [181,] [182,] [183,] [184,] [185,] [186,] [187,] [188,]
"high"
[189,] [190,] [191,] [192,] [193,] [194,] [195,] [196,] [197,] [198,] [199,] [200,] [201,] [202,] [203,] [204,] [205,] [206,] [207,] [208,] [209,]
"high"
[210,] [211,] [212,] [213,] [214,] [215,] [216,] [217,] [218,] [219,] [220,] [221,] [222,] [223,] [224,] [225,] [226,] [227,] [228,] [229,]
"high" "high" "medium" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "medium" "high" "medium" "high"
[230,] [231,] [232,] [233,] [234,] [235,] [236,] [237,] [238,] [239,] [240,] [241,] [242,] [243,] [244,] [245,] [246,] [247,] [248,] [249,] [250,]
"high"
[251,] [252,] [253,] [254,] [255,] [256,] [257,] [258,] [259,] [260,] [261,] [262,] [263,] [264,] [265,] [266,] [267,] [268,] [269,] [270,] [271,]
"high"
[272,] [273,] [274,] [275,] [276,] [277,] [278,] [279,] [280,] [281,] [282,] [283,] [284,] [285,] [286,] [287,] [288,] [289,] [290,] [291,] [292,]
"high"
[293,] [294,] [295,] [296,] [297,] [298,] [299,] [300,] [301,] [302,] [303,] [304,] [305,] [306,] [307,] [308,] [309,] [310,] [311,] [312,] [313,]
"high"
[314,] [315,] [316,] [317,] [318,] [319,] [320,] [321,] [322,] [323,] [324,] [325,] [326,] [327,] [328,] [329,] [330,] [331,] [332,] [333,]
"medium" "high" "high" "high" "medium" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high"
[334,] [335,] [336,] [337,] [338,] [339,] [340,] [341,] [342,] [343,] [344,] [345,] [346,] [347,] [348,] [349,] [350,] [351,] [352,] [353,] [354,]
"high"
[355,] [356,] [357,] [358,] [359,] [360,] [361,] [362,] [363,] [364,] [365,] [366,] [367,] [368,] [369,] [370,] [371,] [372,] [373,] [374,] [375,]
"high"
[376,] [377,] [378,] [379,] [380,] [381,] [382,] [383,] [384,] [385,] [386,] [387,] [388,] [389,] [390,] [391,] [392,] [393,] [394,] [395,] [396,]
"high" "high" "high" "high" "high" "high" "medium" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high"
[397,] [398,] [399,] [400,] [401,] [402,] [403,] [404,] [405,] [406,] [407,] [408,] [409,] [410,] [411,] [412,] [413,] [414,] [415,] [416,] [417,]
"high"
[418,] [419,] [420,] [421,] [422,] [423,] [424,] [425,] [426,] [427,] [428,] [429,] [430,] [431,] [432,] [433,] [434,] [435,] [436,] [437,] [438,]
"medium" "high" "high" "high" "high" "high" "high" "high" "high" "high" "high" "low" "high" "medium" "high" "high" "high" "high" "high" "high" "high" "high"
[439,] [440,] [441,] [442,] [443,] [444,] [445,] [446,] [447,] [448,] [4

2. 哪一個 peak category (尖峰類別) 的 High 品質基因比例最高？

```
## 用 list 存有哪些類別，這些類別有幾個 High 品質基因

peak_category <- list()
for (i in 1:nrow(data)){
  category <- data[i, ncol(data)] # 現在的 peak category
  if (!(data[i, ncol(data)] %in% names(peak_category))){ # category 還沒出現過就初始化
    peak_category[[category]] <- c(0, 0) # 向量 1 用來存 high 出現的次數; 2 存總數
  }
  if (quality_list[[i]] == 'high'){ # 計算有幾個 high 品質
    peak_category[[category]][1] <- peak_category[[category]][1] + 1
  }
  # 計算總數
  peak_category[[category]][2] <- peak_category[[category]][2] + 1
}
peak_category
```

Output:

```
$G1
[1] 284 300

$`G2/M`
[1] 185 195

$`M/G1`
[1] 104 113

$S
[1] 66 71

$`S/G2`
[1] 118 121
```

```

# 計算 High 品質基因比例，並回傳最大值的索引
count_max_high_percent_peak_category <- function(lst){
  max_percent = 0
  idx = 1
  for (i in 1:length(lst)){
    if (lst[[i]][1] / lst[[i]][2] > max_percent){
      max_percent <- lst[[i]][1] / lst[[i]][2]
      idx <- i
    }
  }
  return(idx)
}
# 找到 High 品質基因比例最高的類別索引
max_high_percent_peak_category <-
count_max_high_percent_peak_category(peak_category)
# 輸出類別
names(peak_category)[max_high_percent_peak_category]

```

Output:

```
[1] "S/G2"
```

加分題 B

SRBCT_train.txt 的樣本分屬四個類別 (EWS / BL / NB / RMS，由欄名前綴判斷)。

請對每個 gene，分別算出它在每一類別的 mean 與 sd。

也就是產生一個「gene × 類別」的 mean 表與 sd 表 (各 4 欄，對應 4 類別)。

AI

先把所有名稱用.分離 eg:EWS.T1 -> "EWS" "T1"

```
SRBCT_col_name <- strsplit(names(data2[,c(-1,-2)]), "\\.")
```

R 有跟 python 的 split() 一樣的函式嗎

中午12:01

釐清 R 的字串分割函式與 Python 差異。

有,R 裡最接近的是 `strsplit()`。不過有兩個重要差異要注意。

回傳的是 **list**,不是向量。因為 `strsplit()` 可以一次處理多個字串,所以它把結果包在 list 裡。
單一字串通常要用 `[[1]]` 取出來:

```
r
strsplit("a,b,c", ",")
# [[1]]
# [1] "a" "b" "c"

strsplit("a,b,c", ",")[[1]]
# [1] "a" "b" "c"
```

分隔符號預設是「正規表達式」(regex),不是純文字。這跟 Python 的 `split()` 用字面比對不同。所以遇到 regex 特殊字元(像 `.`、`|`、`*`)會出問題:

```
r
strsplit("a.b.c", ".")[[1]]
# [1] "" "" "" "" "" "" <- "." 被當成「任意字元」

strsplit("a.b.c", ".", fixed = TRUE)[[1]]
# [1] "a" "b" "c" <- 加 fixed = TRUE 就照字面切
```

另外,Python 的 `"a b c".split()` 不給參數時會用空白切並去掉空字串,R 沒有這個預設,要自己指定 pattern(例如 `"\\s+"`)。

如果你有用 tidyverse, `stringr::str_split()` 用起來也很像,而且行為比較一致:

```
r
library(stringr)
str_split("a,b,c", ",") # 一樣回傳 list
str_split_1("a,b,c", ",") # 直接回傳向量,不用 [[1]]
```

簡單來說:一般情況用 `strsplit(x, sep)[[1]]`,分隔符號有特殊字元記得加 `fixed = TRUE`。

思路：找到類別起始與終止的索引，我就可以用[起始：終止]算 mean、sd

```
## 我用來存放每個類別的起始與終止的索引
four_category_start_end <- list(EWS = c(0, 0), BL = c(0, 0), NB = c(0, 0), RMS = c(0, 0))
## 存放四類別的平均值與標準差
four_category_mean_sd <- array(vector("list", nrow(data2_without_ch) * 4), dim = c(nrow(data2_without_ch), 4))

# 尋找每個類別的起始與終止的索引
for (i in 1:nrow(data2_without_ch)){
  if (SRBCT_col_name[[i]][1] == "EWS" & four_category_start_end[["EWS"]][1] == 0){
    four_category_start_end[["EWS"]][1] <- i
  }else if (SRBCT_col_name[[i]][1] == "EWS"){
    four_category_start_end[["EWS"]][2] <- i
  }
  if (SRBCT_col_name[[i]][1] == "BL" & four_category_start_end[["BL"]][1] == 0){
    four_category_start_end[["BL"]][1] <- i
  }else if (SRBCT_col_name[[i]][1] == "BL"){
    four_category_start_end[["BL"]][2] <- i
  }
  if (SRBCT_col_name[[i]][1] == "NB" & four_category_start_end[["NB"]][1] == 0){
    four_category_start_end[["NB"]][1] <- i
  }else if (SRBCT_col_name[[i]][1] == "NB"){
    four_category_start_end[["NB"]][2] <- i
  }
  if (SRBCT_col_name[[i]][1] == "RMS" & four_category_start_end[["RMS"]][1] == 0){
    four_category_start_end[["RMS"]][1] <- i
  }else if (SRBCT_col_name[[i]][1] == "RMS"){
    four_category_start_end[["RMS"]][2] <- i
  }
}
```

計算平均值與標準差

```
for (i in 1:nrow(data2_without_ch)){
  for (j in 1:4){
    strat_idx <- four_category_start_end[[j]][1]
    end_idx <- four_category_start_end[[j]][2]
    m <- mean(data2_without_ch[i, strat_idx : end_idx])
    s <- sd(data2_without_ch[i, strat_idx : end_idx])
    four_category_mean_sd[[i, j]] <- c(m, s)
  }
}
```

AI

```
# 存成表
mean_table <- data.frame("gene" = data2[, 1],
  'EWS' = sapply(four_category_mean_sd[, 1], function(x) x[1]),
  'BL' = sapply(four_category_mean_sd[, 2], function(x) x[1]),
  'NB' = sapply(four_category_mean_sd[, 3], function(x) x[1]),
  'RMS' = sapply(four_category_mean_sd[, 4], function(x) x[1]),
  row.names = NULL)

# 存成表
sd_table <- data.frame("gene" = data2[, 1],
  'EWS' = sapply(four_category_mean_sd[, 1], function(x) x[2]),
  'BL' = sapply(four_category_mean_sd[, 2], function(x) x[2]),
  'NB' = sapply(four_category_mean_sd[, 3], function(x) x[2]),
  'RMS' = sapply(four_category_mean_sd[, 4], function(x) x[2]),
  row.names = NULL)
```

Output:

	gene	EWS	BL	NB	RMS
1	21652.000	1.9041609	0.2309625	1.0245250	1.827855
2	25725.000	0.1329913	0.0821625	0.1360750	0.633335
3	26184.000	1.0280609	1.1280375	1.4801583	0.546105
4	22260.000	0.4354826	0.7364125	1.0880500	0.517555
5	22293.000	0.3333435	0.3142375	0.1988000	0.320170
6	22493.000	2.3626652	1.4348500	1.5085250	2.435905
7	23019.000	4.7450000	3.9033250	4.9140167	6.026830
8	23132.000	1.3242391	1.3484750	0.7911167	1.340385
9	24145.000	1.0131783	1.8612250	1.6624167	1.173540
10	25584.000	1.3994435	2.1797500	1.4279500	1.639000
11	31143.000	5.0576609	4.8701125	5.0379333	6.466745
12	31169.000	0.7453217	1.0093375	0.9363917	0.879345
13	27549.000	4.8238870	4.9903625	5.1367250	5.230675
14	27624.000	1.8320217	1.2650125	1.3901417	2.125590
15	27848.000	1.1738609	1.1405500	0.7739250	1.148370
16	28410.000	0.8389739	0.4883625	0.6514000	0.752805
17	29054.000	1.1052565	0.7279875	1.4254333	0.961695
18	34945.000	2.5590913	2.7513625	2.2197333	1.942395
19	35483.000	1.1267696	1.0434750	1.3235333	0.928305
20	32875.000	1.2702304	1.5458000	1.0450500	1.229645
21	32898.000	1.6495957	1.0689375	1.3848917	1.892815
22	33525.000	5.9149261	5.4823000	4.6596583	6.790380
23	33632.000	0.5323435	0.3845500	0.4893333	0.552250
24	34357.000	3.5869435	4.5004875	4.8436417	4.255005
25	34355.000	2.6132609	3.9533750	4.6060417	2.733920
26	34396.000	4.4409000	4.4648750	4.7305167	6.218095
27	36950.000	1.4666826	1.0057500	0.4465750	1.171245
28	39285.000	1.3970826	1.9041500	1.3371500	1.117455
29	39796.000	0.5748696	0.3185375	0.1616667	0.325730
30	37196.000	1.4771783	0.6200000	1.2592667	1.151970
31	37904.000	0.5252304	0.4206250	0.6140083	0.684930
32	38471.000	1.3542174	0.6478875	0.6685000	1.049885
33	39093.000	1.0227739	0.0897250	0.1043667	0.890290
34	39993.000	1.8970261	3.0852625	2.5537667	2.020085
35	42352.000	3.7490870	4.2617000	3.4605000	3.633875
36	42576.000	2.9189217	1.6447750	1.5879000	1.552610

Showing 1 to 36 of 2,308 entries, 5 total columns

	gene	EWS	BL	NB	RMS
1	21652.000	0.87228254	0.11974824	0.36531413	0.71810563
2	25725.000	0.05675800	0.03083523	0.05275325	0.48915916
3	26184.000	0.57001903	0.55695955	0.84721229	0.30782117
4	22260.000	0.48071184	0.61087578	0.62827364	0.55542126
5	22293.000	0.12838979	0.19108604	0.13643195	0.20866079
6	22493.000	2.16197344	0.69038087	0.64434438	1.83417262
7	23019.000	1.96902081	0.67400313	1.16416554	3.17686448
8	23132.000	0.62141953	0.63173145	0.29070926	0.54103152
9	24145.000	0.27457112	0.53726005	0.99751352	0.38663435
10	25584.000	0.66978234	0.58397787	0.40250836	0.77449678
11	31143.000	1.56417527	0.63420318	1.16772061	3.04665628
12	31169.000	0.38158567	0.28699824	0.21873882	0.41206627
13	27549.000	1.70508113	0.66352235	1.20644090	2.35509962
14	27624.000	0.88369142	0.50064798	0.42225857	0.83712728
15	27848.000	0.46243814	0.77993575	0.43814160	0.65687193
16	28410.000	0.53144812	0.13846585	0.15981707	0.52010626
17	29054.000	0.37566955	0.10645363	0.39697580	0.42050219
18	34945.000	1.17570665	0.88041546	0.88979134	0.78292692
19	35483.000	1.09687366	0.61244740	0.73614603	0.83739239
20	32875.000	0.72948004	0.80671343	0.45595060	0.55480558
21	32898.000	0.65398753	0.53616364	0.61307621	0.80332238
22	33525.000	1.55208191	0.95614381	0.81382929	2.68707799
23	33632.000	0.33701817	0.15208775	0.22623422	0.23239534
24	34357.000	1.33940585	1.00085415	1.20206647	2.72432415
25	34355.000	1.19044831	0.67414801	1.36339361	1.66219215
26	34396.000	1.11532987	0.45373884	0.79745828	2.06639842
27	36950.000	0.69816856	0.72187935	0.19038476	0.73520859
28	39285.000	0.90193188	0.77263642	0.49022280	0.51051921
29	39796.000	0.24670608	0.15574124	0.07931277	0.17555645
30	37196.000	0.97541453	0.25146922	0.53964843	0.61317769
31	37904.000	0.27297717	0.14839955	0.34069385	0.24886368
32	38471.000	0.66417879	0.23161113	0.21828786	0.42948360
33	39093.000	0.94655012	0.01819409	0.06769895	0.97186085
34	39993.000	0.55698462	0.86514273	0.53197687	1.03256599
35	42352.000	1.53810268	1.60793586	1.12896877	1.24644537
36	42576.000	1.25692459	0.54261286	0.51689143	0.57233525

Showing 1 to 36 of 2,308 entries, 5 total columns

```
# 計算平均值與標準差
for (i in 1:nrow(data2_without_ch)){
  for (j in 1:4){
    strat_idx <- four_category_start_end[[j]][1]
    end_idx <- four_category_start_end[[j]][2]
    m <- mean(data2_without_ch[i, strat_idx : end_idx])
    s <- sd(data2_without_ch[i, strat_idx : end_idx])
    four_category_mean_sd[[i, j]] <- c(m, s)
  }
}
```

我想要取出four_category_mean_sd第一欄的所有m值

下午1:29

識別了列表矩陣結構並規劃向量提取策略。

你的 `four_category_mean_sd` 是一個 list-matrix,每個格子存的是 `c(m, s)`。要取出第一欄所有的 `m` 值(也就是每個向量的第一個元素),可以用:

```
r
m_values <- sapply(four_category_mean_sd[, 1], `[, 1])
```

說明:

- `four_category_mean_sd[, 1]` 取出第一欄,回傳一個 list,每個元素是 `c(m, s)`。
- ``[, 1]`` 搭配 `1` 表示對每個元素取第一個位置,也就是 `m`。

如果想更嚴謹地保證回傳型別是數值向量,可以用 `vapply`:

```
r
m_values <- vapply(four_category_mean_sd[, 1], function(x) x[1], numeric(1))
```

兩種寫法結果一樣,都會得到一個長度為 `nrow(data2_without_ch)` 的數值向量。若之後還要取 `s` (標準差),把 `1` 改成 `2` 即可。

加分題 C

對 `cycleData_peak.txt`：用 `apply + is.na` 算出每一欄的 NA 個數，
再用 `which.max()` 找出 NA 最多的那一欄是哪一欄。

```
count_NA <- function(x){  
  sum(is.na(x))  
}  
  
NA_nums <- apply(data, 2, count_NA)  
max_NA_col <- which.max(NA_nums)  
max_NA_col
```

Output:

```
cdc28.140  
62
```